Backend Summary

Study Group Finder API

Technical Reference Document

What It Is

A **REST API** built with Node.js and Express that powers the Study Group Finder application. It handles user authentication, session management, RSVPs, location tracking, and in-session messaging — storing all data in a MySQL database.

Property	Value
Runtime	Node.js (CommonJS modules)
Framework	Express 5
Database	MySQL via `mysql2/promise` connection pool
Port	`5003` (configurable via `.env`)

How It Starts

```
npm run dev  ->  nodemon src/server.js
```

`server.js` does three things in order:

1. **Loads `.env`** — all secrets and config (DB credentials, JWT secret, port) come from environment variables, never hardcoded.
2. **Opens the DB connection pool** — `config/db.js` is required, which immediately tests the MySQL connection.
3. **Runs `initDB()`** — creates all 5 database tables if they do not exist yet, then starts listening.

> **Note:** If MySQL is unreachable, the server **refuses to start** and exits with code `1` rather than crashing mid-request.

Architecture — The Layers

Every HTTP request travels through these layers in order:

```
HTTP Request
     |
     v
+--------------------------------------+
|           app.js (Express)           |  <- CORS, JSON parsing,
   rate limiting
+------------------+-------------------+
                   |
     +-------------v-------------+
```

```
  |    Route Files             |  <- decides which controller
handles it
  |  authRoutes.js             |
  |  groupRoutes.js            |  <- also handles message
routes
  +-------------+-------------+
                |
  +-------------v-------------+
  |    Middleware              |  <- verifyToken (JWT check,
protected routes)
  |  authMiddleware.js         |
  +-------------+-------------+
                |
  +-------------v-------------+
  |    Controllers             |  <- validates input, applies
business rules
  |  authController.js         |
  |  groupController.js        |
  |  messageController.js      |
  +-------------+-------------+
                |
  +-------------v-------------+
  |    Models                  |  <- executes parameterised
SQL queries
  |  User.js                   |
  |  Group.js  (Session)       |
  |  Location.js               |
  |  Message.js                |
  +-------------+-------------+
                |
  +-------------v-------------+
  |    MySQL Database          |  <- stores all persistent
data
  |  users                     |
  |  locations                 |
  |  sessions                  |
  |  session_attendees         |
  |  messages                  |
  +----------------------------+
                |
  +-------------v-------------+
  |    errorHandler.js         |  <- catches any error,
returns clean JSON
  +----------------------------+
```

The Database — All 5 Tables

Tables are created in dependency order so foreign keys never reference a table that does not exist yet.

1. `users`

Stores registered accounts.

Column	Type	Notes
`id`	`INT PK`	Auto-increment
`username`	`VARCHAR(100)`	Display name
`email`	`VARCHAR(100) UNIQUE`	Used for login
`password`	`VARCHAR(255)`	bcrypt hash
`created_at`	`TIMESTAMP`	

2. `locations`

Stores physical or virtual meeting places, decoupled from sessions so a place can be reused.

Column	Type	Notes
`id`	`INT PK`	
`address`	`VARCHAR(255)`	Text address (required)
`place_name`	`VARCHAR(255)`	Optional label e.g. “Main Library 3F”
`lat`	`DECIMAL(9,6)`	Nullable — map pin latitude
`lng`	`DECIMAL(9,6)`	Nullable — map pin longitude
`created_at`	`TIMESTAMP`	

3. `sessions`

Stores every study session. References `users` (host) and `locations`.

Column	Type	Notes
`id`	`INT PK`	
`tag`	`VARCHAR(50)`	Course code e.g. `COMP 401`
`title`	`VARCHAR(200)`	Session headline
`time`	`VARCHAR(100)`	Human-readable e.g. `Friday, 4:00 PM`
`about`	`TEXT`	Nullable description
`total_spots`	`INT`	Max attendees, default 6
`color`	`VARCHAR(20)`	Map marker hex colour
`host_id`	`INT FK → users`	`ON DELETE CASCADE`
`location_id`	`INT FK → locations`	`ON DELETE RESTRICT`
`created_at`	`TIMESTAMP`	

4. `session_attendees` (RSVP)

Join table that tracks who joined which session.

Column	Type	Notes
session_id	INT PK, FK → sessions	ON DELETE CASCADE
user_id	INT PK, FK → users	ON DELETE CASCADE
joined_at	TIMESTAMP	

Composite PK (session_id, user_id) prevents duplicate joins at the DB level.

5. messages

Stores the discussion thread for each session.

Column	Type	Notes
id	INT PK	
session_id	INT FK → sessions	ON DELETE CASCADE
user_id	INT FK → users	ON DELETE CASCADE
body	TEXT	Message content (required)
created_at	TIMESTAMP	

All API Endpoints

Auth — /api/auth

Method	Endpoint	Auth	What it does
POST	/api/auth/register	No	Create a new account
POST	/api/auth/login	No	Log in, receive a JWT token
POST	/api/auth/logout	No	Stateless logout instruction

Sessions — /api/sessions

Method	Endpoint	Auth	What it does
GET	/api/sessions	No	All sessions with location + attendee count
GET	/api/sessions/:id	No	One session with location + full attendees list
POST	/api/sessions	JWT	Create location + session in one request
POST	/api/sessions/:id/join	JWT	RSVP to a session
DELETE	/api/sessions/:id/leave	JWT	Cancel RSVP
DELETE	/api/sessions/:id	JWT	Delete session (host only)

Messages — `/api/sessions/:id/messages`

Method	Endpoint	Auth	What it does
`GET`	`/api/sessions/:id/messages`	No	Get all messages for a session
`POST`	`/api/sessions/:id/messages`	JWT	Post a message
`DELETE`	`/api/sessions/:id/messages/:msgId`	JWT	Delete own message

Utility

Method	Endpoint	Auth	What it does
`GET`	`/api/health`	No	Check the server is running
`GET`	`/`	No	API welcome message

How Each Operation Works

Register

1. Joi validates the body (`username`, `email`, `password`).

2. Checks the DB for a duplicate email — rejects if found.

3. bcrypt hashes the password (salt rounds = 12).

4. Inserts the user into MySQL.

5. Returns the new user (no password in the response).

```
POST /api/auth/register
{ "username": "Nedim", "email": "nedim@unic.ac.cy", "password":
   "secret123" }

-> 201 { "data": { "id": 1, "username": "Nedim", "email": "..."
   } }
```

Login

1. Joi validates the body.

2. Finds the user by email in the DB.

3. `bcrypt.compare()` checks the submitted password against the stored hash.

4. On success, signs a JWT containing `{id, email, username}`, valid for 24 hours.

5. Returns the token — the client stores this and sends it on every protected request.

```
POST /api/auth/login
{ "email": "nedim@unic.ac.cy", "password": "secret123" }

-> 200 { "data": { "token": "eyJ...", "user": { "id": 1, ... } }
    }
```

Protected Request (e.g. Create Session)

Every protected route runs `verifyToken` middleware first:

1. Checks for `Authorization: Bearer <token>` header.
2. `jwt.verify()` decodes and validates the token against `JWT_SECRET`.
3. Attaches decoded user (`{id, email, username}`) to `req.user`.
4. Calls `next()` — the controller runs.

Create Session (Two-Step)

1. Joi validates all fields including location fields.
2. **Step 1:** `Location.create({ address, placeName, lat, lng })` inserts into `locations` and returns `location.id`.
3. **Step 2:** `Session.create({ ..., locationId })` inserts into `sessions` referencing the new location.
4. Returns the full session with its location joined in.

Join Session

1. Verifies the session exists.
2. Rejects if the user is the host (they are already the creator).
3. Checks `total_spots - COUNT(attendees)` — rejects with `409` if full.
4. `INSERT IGNORE` into `session_attendees` (idempotent — joining twice is safe).
5. Returns the updated session.

Post a Message

1. Verifies the session exists.
2. Validates `body` with Joi.
3. Inserts into `messages` with `session_id` and `user_id = req.user.id`.
4. Returns the new message with author username.

Delete Message

1. Runs `DELETE FROM messages WHERE id = ? AND user_id = req.user.id`.
2. If `affectedRows = 0` — returns `403` (not found or not the author).

Delete Session

1. Runs `DELETE FROM sessions WHERE id = ? AND host_id = req.user.id`.
2. If `affectedRows = 0`, the session either does not exist or the user is not the host — returns `403`.

3. Cascade delete in MySQL removes all `session_attendees` and `messages` rows automatically.

Security Measures

Measure	Implementation
Password hashing	`bcryptjs` with 12 salt rounds — passwords never stored or returned as plain text
JWT authentication	Signed with `JWT_SECRET` from `.env`, 24-hour expiry
Rate limiting	Login endpoint: max 10 attempts per 15 minutes (blocks brute-force)
Input validation	Every endpoint validates its body with Joi before touching the DB
SQL injection prevention	All queries use parameterised `?` placeholders — no string concatenation
Secrets in env	All credentials live in `.env`, never in source code
Owner-gated actions	Host-only delete enforced in SQL (`WHERE id = ? AND host_id = ?`)
Author-gated messages	Message delete enforced in SQL (`WHERE id = ? AND user_id = ?`)

Project File Map

```
src/
|-- server.js                    <- entry point
|-- app.js                       <- express setup
|
|-- config/
|   |-- db.js                    <- MySQL connection pool
|   `-- initDB.js                <- CREATE TABLE IF NOT EXISTS (
   all 5 tables)
|
|-- routes/
|   |-- authRoutes.js            <- /api/auth/*
|   `-- groupRoutes.js           <- /api/sessions/* + /api/
   sessions/:id/messages/*
|
|-- controllers/
|   |-- authController.js        <- register, login, logout
|   |-- groupController.js       <- session CRUD + join/leave
|   `-- messageController.js     <- getMessages, sendMessage,
   deleteMessage
|
|-- models/
|   |-- User.js                  <- findByEmail, findById,
   createUser
|   |-- Group.js                 <- session queries (JOINs
   locations)
```

```
|   |-- Location.js              <- create, findById
|   `-- Message.js               <- getBySession, create, delete
|
`-- middleware/
    |-- authMiddleware.js        <- verifyToken
    `-- errorHandler.js         <- global error handler
```

What Is Not Yet Done

Feature	Status
User profile update (`PUT /api/users/me`)	Not implemented
Session update (`PUT /api/sessions/:id`)	Not implemented
Pagination on `GET /api/sessions`	Not implemented
Search / filter sessions by tag or location	Not implemented
Refresh tokens	Not implemented
Front-end ↔ back-end wiring	Not done (front-end uses localStorage)